command. For testing, run the following code:

```
$ emrun -no_browser -port 8080 .

Web server root directory: <your current directory>
Now listening at http://localhost:8080/
```

Open the browser *http://localhost:8080/hello.html* and you should see 'hello world' printed.

WebAssembly and Rust

Rust is a programming language sponsored by Mozilla Research. It's used for creating highly concurrent and safe systems. The syntax is similar to that of C/C++.

Since Firefox uses Rust, it seemed natural that it should be possible to program in Rust and compile to WebAssembly. You may follow the steps given at *https://goo.gl/LPIL8B* to install and test compiling Rust code to WebAssembly.

Rust is available in many repositories; however, you will need to use the *rustup* installer from *https://www.rustup.rs/* to install the compiler in your local environment and then add the modules needed for WebAssembly as follows:

```
$ curl https://sh.rustup.rs -sSf | sh
$ source ~/.cargo/env
$ rustup target add asmjs-unknown-emscripten
$ rustup target add wasm32-unknown-emscripten
```

You may now write your first Rust program, *hello.rs*, as follows:

```
fn main(){
    println!("Hello, Emscripten!");
}
```

Compile and run the program and verify that you get the expected output:

```
$ rustc hello.rs
$ hello
Hello, Emscripten!
```

Now, you may compile to JavaScript and test it as follows:

```
$ rustc --target=asmjs-unknown-emscripten hello.rs
$ node hello.js
Hello, Emscripten!
```

You can create the *wasm* target and an *HTML* front:

```
$ rustc --target=wasm32-unknown-emscripten hello.rs -o
hello.html
```

You can test it as with the C example, as follows:

```
$ emrun -no_browser -port 8080 .
Web server root directory: <your current directory>
Now listening at http://localhost:8080/
```

Why bother?

The importance of these projects cannot be underestimated. JavaScript has become very popular and, with Node.js, on the server side as well.

There is still a need to be able to write secure and reliable Web applications even though the growth of mobile apps has been explosive.

It would appear that mobile devices and 'apps' are taking over; however, there are very few instances in which the utility of an app is justified. In most cases, there is no reason that the same result cannot be achieved using the browser. For example, I do not find a need for the Facebook app. Browsing *m.facebook.com* is a perfectly fine experience.

When an e-commerce site offers me a better price if I use its app on a mobile device, it makes me very suspicious. The suspicions seem all too often to be justified by the permissions sought by many of the apps at the time of installation. Since it is hard to know which app publisher to trust, I prefer finding privacy-friendly apps, e.g., *https://goo.gl/aUmns3*.

Coding complex applications in JavaScript is hard. FirefoxOS may not have succeeded, but given the support by all the major browser developers, the future of WebAssembly should be bright. You can be sure that tools like Emscripten will emerge for even more languages, and you can expect apps to lose their importance in favour of the far safer and more trustworthy WebAssembly code. END

By: Dr Anil Seth

The author has earned the right to do what interests him. You can find him online at *http://sethanil.com*, *http://sethanil.blogspot.com*, and reach him via email at *anil@sethanil.com*.